

QUESTION 9

LOCAL RETRIEVAL-AUGMENTED GENERATION (RAG) SYSTEM

Implementation using Engineering Documents, ChromaDB Vector Database and Ollama

1. Question

Implement a local Retrieval-Augmented Generation (RAG) system using engineering documents, a vector database, and Ollama to answer technical questions.

2. Aim

To develop a local RAG system that retrieves relevant information from engineering documents using vector similarity search and uses the locally running Llama 3.2 model through Ollama to generate grounded technical answers.

3. Objectives

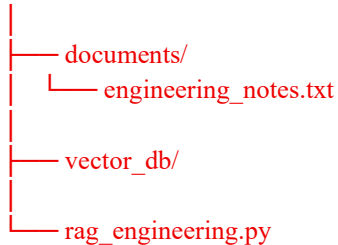
- Store engineering knowledge in local technical documents.
- Divide documents into smaller searchable chunks.
- Convert document chunks and questions into embeddings.
- Store and retrieve relevant information using ChromaDB.
- Use Ollama with Llama 3.2 to generate answers from retrieved context.
- Reduce unsupported answers by grounding the response in reference information.

4. Technologies and Tools Used

Technology / Tool	Purpose
Python	RAG pipeline implementation
Ollama	Local LLM runtime
Llama 3.2	Local model for answer generation
ChromaDB	Vector database
Sentence Transformers	Text embedding generation
all-MiniLM-L6-v2	Embedding model
VS Code	Development environment

5. Project Structure

Local_LLM_Text_Generation



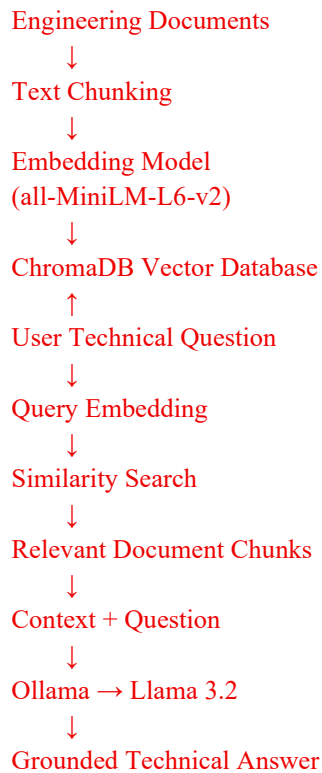
6. Engineering Document

The local file `engineering_notes.txt` contains information about temperature monitoring, vibration monitoring, preventive maintenance, electrical safety, troubleshooting, and cooling systems.

7. Algorithm / Procedure

1. Start the RAG application and load the embedding model.
2. Read engineering documents from the documents folder.
3. Divide the document into overlapping text chunks.
4. Generate embeddings for each chunk.
5. Create or open the persistent ChromaDB vector database.
6. Store chunks, embeddings, and source information.
7. Accept a technical question from the user.
8. Generate an embedding for the question.
9. Retrieve the three most relevant chunks using vector similarity.
10. Combine the retrieved information with the question.
11. Send the grounded prompt to Llama 3.2 through Ollama.
12. Display the retrieved sources and generated technical answer.

8. System Architecture



9. Implementation Code

```
import os
import chromadb
import ollama
from sentence_transformers import SentenceTransformer

DOCUMENT_FOLDER = "documents"
VECTOR_FOLDER = "vector_db"
MODEL_NAME = "llama3.2"
EMBEDDING_MODEL = "all-MiniLM-L6-v2"

print("=" * 70)
print("LOCAL ENGINEERING RAG SYSTEM")
print("=" * 70)

print("\nLoading embedding model...")
embedding_model = SentenceTransformer(EMBEDDING_MODEL)
print("Embedding model loaded successfully.")
```

```

documents = []
for filename in os.listdir(DOCUMENT_FOLDER):
    file_path = os.path.join(DOCUMENT_FOLDER, filename)
    if filename.endswith(".txt"):
        with open(file_path, "r", encoding="utf-8") as file:
            text = file.read()
            documents.append({"name": filename, "text": text})

if not documents:
    print("No engineering documents found.")
    exit()

print(f'\nLoaded {len(documents)} engineering document(s).')

chunks = []
chunk_size = 800
overlap = 100

for document in documents:
    text = document["text"]
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end].strip()
        if chunk:
            chunks.append({"text": chunk, "source": document["name"]})
        start = end - overlap

print(f'\nCreated {len(chunks)} document chunks.')

print("\nCreating embeddings...")
texts = [chunk["text"] for chunk in chunks]
embeddings = embedding_model.encode(texts).tolist()
print("Embeddings created successfully.")

client = chromadb.PersistentClient(path=VECTOR_FOLDER)
collection = client.get_or_create_collection(name="engineering_documents")

ids = [f'chunk_{i}' for i in range(len(chunks))]
metadatas = [{"source": chunk["source"]} for chunk in chunks]

collection.upsert(
    ids=ids,
    documents=texts,
    embeddings=embeddings,
    metadatas=metadatas
)

print("Engineering documents stored in vector database.")

question = input("\nEnter your technical question: ")
query_embedding = embedding_model.encode([question]).tolist()[0]

results = collection.query(
    query_embeddings=[query_embedding],
    n_results=3

```

```

)

retrieved_documents = results["documents"][0]
retrieved_sources = results["metadatas"][0]
context = "\n\n".join(retrieved_documents)

prompt = f"""
You are an engineering technical assistant.
Answer the user's question using only the provided
engineering reference information.
Do not invent technical facts.
If the answer is not available, say:
"Information not available in the provided documents."

Engineering Reference Information:
{context}

User Question:
{question}

Provide a clear and concise technical answer.
"""

print("\nGenerating answer using Ollama...")

response = ollama.chat(
    model=MODEL_NAME,
    messages=[{"role": "user", "content": prompt}]
)

answer = response["message"]["content"]

print("\n" + "=" * 70)
print("RETRIEVED INFORMATION")
print("=" * 70)

for i, source in enumerate(retrieved_sources):
    print(f"{i + 1}. {source['source']}")

print("\n" + "=" * 70)
print("RAG GENERATED ANSWER")
print("=" * 70)
print(answer)

print("\n" + "=" * 70)
print("RAG PROCESS COMPLETED")
print("=" * 70)

```

10. Execution Commands

```
pip install chromadb
```

```
pip install sentence-transformers
```

```
pip install pypdf
```

```
python rag_engineering.py
```

11. Actual Execution Output

LOCAL ENGINEERING RAG SYSTEM

Loading embedding model...

Embedding model loaded successfully.

Loaded 1 engineering document(s).

Created 4 document chunks.

Creating embeddings...

Embeddings created successfully.

Engineering documents stored in vector database.

Enter your technical question:

What can cause increasing vibration levels in rotating machinery?

Generating answer using Ollama...

RETRIEVED INFORMATION

1. engineering_notes.txt
2. engineering_notes.txt
3. engineering_notes.txt

RAG GENERATED ANSWER

According to the Engineering System Maintenance Notes, increasing vibration levels in rotating machinery may indicate mechanical imbalance, misalignment, bearing problems, or loose components.

RAG PROCESS COMPLETED

12. Result Analysis

Item	Result
Engineering documents loaded	1
Document chunks created	4
Embeddings	Successfully generated
Vector database	ChromaDB
Retrieved chunks	3
Local LLM	Llama 3.2 through Ollama
Technical question	What can cause increasing vibration levels in rotating machinery?
Answer	Mechanical imbalance, misalignment, bearing problems, or loose components

13. Advantages

- Runs locally using Ollama without requiring a cloud LLM API.
- Uses engineering-specific documents as the knowledge source.
- Semantic vector search retrieves relevant information.
- The knowledge base can be expanded with additional local documents.
- Grounded prompting helps reduce unsupported responses.

14. Conclusion

The local engineering RAG system was successfully implemented using Python, Sentence Transformers, ChromaDB, and Ollama with Llama 3.2. The system loaded one engineering document, divided it into four chunks, generated embeddings, stored them in a vector database, retrieved three relevant chunks, and generated a grounded technical answer. The successful execution demonstrates how local document retrieval can be combined with a locally running Large Language Model for technical question answering.

15. Short Explanation for Presentation

This project is a local RAG-based technical question-answering system. Engineering documents are divided into chunks and converted into embeddings. The embeddings are stored in ChromaDB. When a user asks a question, the system retrieves the most relevant chunks and sends them with the question to Llama 3.2 through Ollama. The model generates a technical answer based on the retrieved information.